

## Graphs

### Fundamentals

- A **graph** consists of a set of vertices (or nodes) and a set of edges (relations) between the pairs of vertices.
- The edges represent paths or connections between the vertices. Each edge is a set of two (2) vertices.
- Example of a graph:

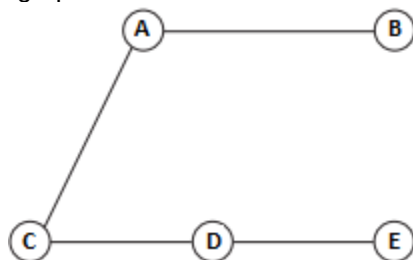


Figure 1. Undirected graph

Based on the above graph, the sets of vertices (V) and edges (E) are as follows:

$$V = \{A, B, C, D, E\}$$

$$E = \{\{A, B\}, \{A, C\}, \{C, D\}, \{D, E\}\}$$

- A **digraph** (directed graph) consists of edges each connected to a single vertex. Directed edges are lines with an arrow on one (1) end.

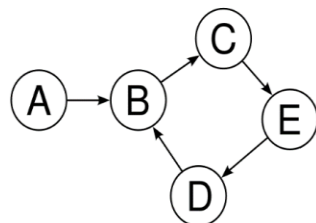


Figure 2. Directed graph

Hence, the set of edges is:

$$E = \{\{A, B\}, \{B, C\}, \{C, E\}, \{E, D\}, \{D, B\}\}$$

- A **weighted graph** consists of edges that are associated with values.
- A vertex is **adjacent** to another vertex if there is an edge to it from that other vertex.

Based on Figure 1, (1) **A** is adjacent to **B** and **C**; (2) **C** is adjacent to **A** and **D**; and (3) **D** is adjacent to **C** and **E**.

- The **degree** of a vertex is the number of edges which has that vertex as an endpoint.
- Sample table of degrees (based on Figure 1):

| Vertex | Degree |
|--------|--------|
| A      | 2      |
| B      | 1      |
| C      | 2      |
| D      | 2      |
| E      | 1      |

- A **path** is a sequence of adjacent vertices. Based on Figure 2, the path from **A** to **D** is **A-B-C-E-D**.

### Graph Representation

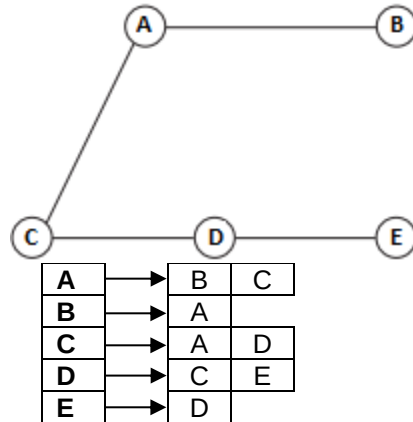
- An **adjacency matrix** is a square matrix with dimensions equivalent to the number of vertices in the graph. It is used to indicate whether pairs of vertices are adjacent or not.
- The elements of the matrix typically have values '0' or '1'. A value of '1' indicates adjacency between the vertices in the row and column.
- Adjacency matrix for an undirected graph (based on Figure 1):

|   | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 0 | 0 |
| B | 1 | 0 | 0 | 0 | 0 |
| C | 1 | 0 | 0 | 1 | 0 |
| D | 0 | 0 | 1 | 0 | 1 |
| E | 0 | 0 | 0 | 1 | 0 |

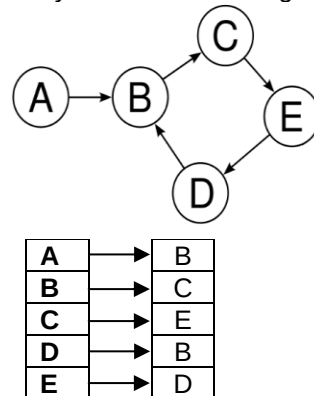
- Adjacency matrix for a directed graph (based on Figure 2):

|   | A | B | C | D | E |
|---|---|---|---|---|---|
| A | 0 | 1 | 0 | 0 | 0 |
| B | 0 | 0 | 1 | 0 | 0 |
| C | 0 | 0 | 0 | 0 | 1 |
| D | 0 | 1 | 0 | 0 | 0 |
| E | 0 | 0 | 0 | 1 | 0 |

- An **adjacency list** uses an array of lists to represent a graph. There is one (1) list for each vertex.
- Example of adjacency list for undirected graph:



- Example of adjacency list for a directed graph:



- Sample code for creating an adjacency list in Java (based on Figure 1):

```
//a list consisting of lists
ArrayList<ArrayList> adjList = new ArrayList<>();
char[] vertices = {'A', 'B', 'C', 'D', 'E'};
int num = 5; //number of vertices
for(int i = 0; i < num; i++) {
    adjList.add(new ArrayList<>());
}
adjList.get(0).add('B');
adjList.get(0).add('C');
adjList.get(1).add('A');
adjList.get(2).add('A');
adjList.get(2).add('D');
adjList.get(3).add('C');
adjList.get(3).add('E');
adjList.get(4).add('D');

int v = 0; //for displaying letters A to E
//for displaying content of each list
for(int i = 0; i < num; i++, v++) {
    System.out.println(vertices[v] + ": " + adjList.get(i));
}
```

#### Output:

```
A: [B, C]
B: [A]
C: [A, D]
D: [C, E]
E: [D]
```

#### Reference:

Koffman, E. & Wolfgang, P. (2016). *Data structures: Abstraction and design using Java*. Hoboken: John Wiley & Sons, Inc.  
 Oracle Docs (n.d.). *Citing sources*. Retrieved from <https://docs.oracle.com/javase/8/docs/api/java/util/package-summary.html>